অধ্যায় ১২

১২

Modern JavaScript (ES6+), Modules এবং Clean Code Practices

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- ES2015 থেকে আজ পর্যন্ত JavaScript-এ যুক্ত হওয়া প্রধান সুবিধাগুলো চিনতে, এবং কোনগুলো আগের অধ্যায়ে শিখেছেন তা মেলাতে।
- `toSorted`, `with`, Logical Assignment, `structuredClone` ও `Object.groupBy`-এর মতো নতুন সুবিধা ব্যবহার করতে।
- ES Module দিয়ে Code-কে আলাদা File-এ ভাগ করতে: Named ও Default `export`, `import`, নাম বদলানো এবং Namespace Import।
- Module-এর নিয়ম বুঝতে: নিজস্ব Scope, স্বয়ংক্রিয় Strict Mode, একবার চলা, Live Binding এবং Dynamic `import()`।
- Browser ও Node.js-এ Module চালাতে, এবং দুই পরিবেশের পার্থক্য সামলাতে।
- পাঠযোগ্য Code লিখতে: অর্থবহ নাম, ছোট Function, Guard Clause, নামযুক্ত Constant।
- Pure Core ও Side Effect-এর সীমানা মেনে Module সাজাতে।
- একটি সম্পূর্ণ **Modular Shopping Cart** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৫: Function, Arrow Function, Default Parameter, Pure Function।
- অধ্যায় ৭: Destructuring, Spread, Rest, Optional Chaining এবং `??`।
- অধ্যায় ৯: DOM, Event Delegation এবং `localStorage`।
- অধ্যায় ১১: নিজের Error-এর ধরন (`class ... extends Error`)।
- Node.js 22 বা নতুন সংস্করণ এবং একটি আধুনিক Browser।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ৯ থেকে ১১-এর Project-গুলোতে একটি অস্বস্তিকর নিয়ম মানতে হয়েছে: HTML-এ Script-গুলো সঠিক ক্রমে রাখতে হতো, আর এক File-এর সব Function ও Variable অন্য File-এ নিজে থেকেই দেখা যেত। দুটি File-এ একই নামের Variable থাকলে একটি অন্যটিকে নীরবে মুছে দিত। তিনটি File-এ এটি সামলানো যায়; তিনশো File-এ যায় না।

ES Module এই সমস্যার মানসম্মত সমাধান। প্রতিটি File নিজের একটি ঘর: বাইরে থেকে শুধু তা-ই দেখা যায় যা সে স্পষ্টভাবে `export` করে, এবং প্রতিটি File নিজেই বলে দেয় কোন File থেকে কী লাগবে। আধুনিক প্রতিটি JavaScript Project—Browser-এ হোক বা Node.js-এ, React হোক বা Server—Module দিয়ে গঠিত।

Module-এর সঙ্গে আসে আরেকটি দক্ষতা: Code এমনভাবে লেখা যাতে অন্য কেউ (এবং ছয় মাস পরের আপনি) সহজে পড়তে ও বদলাতে পারেন। পেশাদার জীবনে আপনি Code লেখার চেয়ে অনেক বেশি সময় Code পড়বেন।

১২.১ Modern JavaScript: কী বদলেছে

২০১৫ সালের ES2015 (ES6 নামে পরিচিত) সংস্করণ JavaScript-কে আমূল বদলে দেয়, এবং তারপর থেকে প্রতি বছর একটি নতুন সংস্করণ আসে। এই বইয়ে আপনি এর বেশির ভাগ ইতিমধ্যে ব্যবহার করেছেন:

সুবিধা	কোন সংস্করণে	এই বইয়ে
`let`, `const`	ES2015	অধ্যায় ২
Template Literal	ES2015	অধ্যায় ২, ৮
Arrow Function, Default Parameter	ES2015	অধ্যায় ৫
Destructuring, Spread, Rest	ES2015, ES2018	অধ্যায় ৭
Promise; `async` / `await`	ES2015; ES2017	অধ্যায় ১০
Optional Chaining, `??`	ES2020	অধ্যায় ৭
`class`	ES2015	অধ্যায় ১১ (সংক্ষেপে), ১৩
Module (`import` / `export`)	ES2015	এই অধ্যায়

এগুলো আবার শেখানো হবে না। এই অধ্যায়ের মূল বিষয় Module এবং পরিষ্কার Code; তার আগে কয়েকটি নতুন সুবিধা, যেগুলো প্রতিদিনের Code-কে আরও নিরাপদ করে।

> **নোট:** নতুন কোনো সুবিধা ব্যবহারের আগে দেখে নিন আপনার লক্ষ্য-পরিবেশ (ব্যবহারকারীদের Browser, Server-এর Node.js) তা সমর্থন করে কি না। MDN-এর প্রতিটি পাতার শেষে "Browser compatibility" সারণি থাকে। এই অধ্যায়ের সব উদাহরণ Node.js 22 এবং Chrome 155-এ পরীক্ষিত।

১২.২ কয়েকটি নতুন সুবিধা

অধ্যায় ৬-এ দেখেছেন, `sort()` মূল Array বদলে দেয়। ES2023-এ এর বদল-না-করা সংস্করণ এসেছে।

new-features.js · JavaScript — চলবে: Node.js

```
const scores = [72, 95, 58];

console.log(scores.toSorted((a, b) => b - a));
console.log(scores.toReversed());
console.log(scores.with(0, 80));
console.log(scores);

const settings = { theme: '', fontSize: 0 };
settings.theme ||= 'light';
settings.fontSize ??= 16;
settings.language ??= 'bn';
console.log(settings);

console.log(1_500_000 + 250_000);
```

Output

```
[ 95, 72, 58 ]
[ 58, 95, 72 ]
[ 80, 95, 58 ]
[ 72, 95, 58 ]
{ theme: 'light', fontSize: 0, language: 'bn' }
1750000
```

- **`toSorted`**, **`toReversed`**, **`with(index, value)`** নতুন Array ফেরত দেয়; শেষ লাইনে মূল `scores` অপরিবর্তিত। অধ্যায় ৭-এর "নতুন মান তৈরি করুন, পুরোনোটি বদলাবেন না" নীতি এখন এক Method-এই।
- **`a ||= b`** মানে `a` Falsy হলে `b` বসাও; **`a ??= b`** মানে শুধু `null` বা `undefined` হলে। পার্থক্যটি দেখুন: ফাঁকা `theme` বদলেছে, কিন্তু `fontSize`-এর `0` থেকে গেছে, কারণ `0` Falsy হলেও `null` নয় (অধ্যায় ৩)।
- **`1_500_000`**: সংখ্যার মাঝের `_` শুধু পড়ার সুবিধার জন্য; মানে কোনো প্রভাব নেই।

clone-group.js · JavaScript — চলবে: Node.js

```
const order = { id: 7, items: [{ name: 'Pen', qty: 2 }] };

const shallow = { ...order };
const deep = structuredClone(order);
deep.items[0].qty = 5;
shallow.items[0].name = 'Pencil';

console.log(order.items[0]);
console.log(deep.items[0]);
```

```
const products = [
  { name: 'Pen', type: 'stationery' },
  { name: 'Mouse', type: 'electronics' },
  { name: 'Notebook', type: 'stationery' },
];
const byType = Object.groupBy(products, (product) => product.type);

console.log(Object.keys(byType));
console.log(byType.stationery.map((product) => product.name));
```

Output

```
{ name: 'Pencil', qty: 2 }
{ name: 'Pen', qty: 5 }
[ 'stationery', 'electronics' ]
[ 'Pen', 'Notebook' ]
```

- **`structuredClone(value)`** একটি সম্পূর্ণ Deep Copy তৈরি করে, অধ্যায় ৭-এর Shallow Copy-র সীমাবদ্ধতা ছাড়াই। Spread-এর Copy-তে ভেতরের Object ভাগাভাগি হয়েছে (`order` -এর নামও বদলে গেছে), কিন্তু `structuredClone` -এর Copy সম্পূর্ণ আলাদা। এটি Function Copy করতে পারে না।
- **`Object.groupBy(items, fn)`** প্রতিটি জিনিসকে `fn` -এর ফেরত দেওয়া Key অনুযায়ী দলে ভাগ করে; অধ্যায় ৭-এর হাতে লেখা গণনা-Object-এর কাজ এক লাইনে।

১২.৩ Module কেন: Global Scope-এর সমস্যা

সাধারণ `<script>` -এ লেখা প্রতিটি সবচেয়ে বাইরের Function একই Global Scope-এ থাকে। দুটি File-এ একই নাম থাকলে কী হয়, Browser-এ দেখুন:

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Global Scope</title>
</head>
<body>
  <script src="tax.js"></script>
  <script src="discount.js"></script>
  <script src="main.js"></script>
</body>
</html>
```

tax.js · JavaScript — চলবে: Browser

```
function calculate(price) {
  return price * 1.15;
}
```

discount.js · JavaScript — চলবে: Browser

```
function calculate(price) {
  return price * 0.9;
}
```

main.js · JavaScript — চলবে: Browser

```
console.log(`VAT-সহ দাম: ${calculate(1000)}`);
```

Output

```
VAT-সহ দাম: 900
```

`main.js`-এর লেখক VAT-এর হিসাব চেয়েছিলেন, পেয়েছেন ছাড়ের হিসাব। পরে লোড হওয়া `discount.js` আগের `calculate`-কে নীরবে প্রতিস্থাপন করেছে; কোনো Error নেই, কোনো সতর্কতা নেই। তা ছাড়া `main.js` পড়ে বোঝার উপায় নেই `calculate` কোথা থেকে আসছে। Module দুটি সমস্যারই সমাধান করে।

১২.৪ export এবং import

একটি Module হলো একটি File, যা নিজের যা অন্যকে দিতে চায় তা **`export`** করে, এবং অন্যের যা লাগে তা **`import`** করে। Node.js-এ `.mjs` Extension বলে দেয় File-টি একটি ES Module।

tax.mjs · JavaScript — চলবে: Node.js

```
export const VAT_RATE = 0.15;

export function withVat(price) {
  return Math.round(price * (1 + VAT_RATE));
}
```

discount.mjs · JavaScript — চলবে: Node.js

```
export function withDiscount(price, percent) {
  return Math.round(price * (1 - percent / 100));
}

export default function describe(price) {
  return `${price.toLocaleString('en-US')} টাকা`;
}
```

main.mjs · JavaScript — চলবে: Node.js

```
import { withVat, VAT_RATE } from './tax.mjs';
import describe, { withDiscount as applyDiscount } from './discount.mjs';
import * as tax from './tax.mjs';

console.log(`VAT ${VAT_RATE * 100}%: ${describe(withVat(1000))}`);
console.log(`ছাড়ের পর: ${describe(applyDiscount(1000, 10))}`);
console.log(tax.withVat(200), Object.keys(tax));
```

Output

```
VAT 15%: 1,150 টাকা
ছাড়ের পর: 900 টাকা
230 [ 'VAT_RATE', 'withVat' ]
```

লেখার ধরন	অর্থ
`export const X` / `export function f`	Named Export: নির্দিষ্ট নামে দেওয়া
`export default ...`	Default Export: File-এর প্রধান জিনিস; একটি File-এ সর্বোচ্চ একটি
`import { a, b } from './file.mjs'`	Named Import; নাম হুবহু মিলতে হবে
`import { a as b }`	Import-এর সময় নাম বদলানো
`import name from './file.mjs'`	Default Import; যেকোনো নাম দেওয়া যায়
`import * as ns from './file.mjs'`	সব Named Export একটি Object-এ (Namespace)

এখন `main.mjs` পড়লেই বোঝা যায় প্রতিটি নাম কোথা থেকে এসেছে, এবং দুটি File-এ একই নাম থাকলেও সংঘর্ষ হয় না: প্রতিটি File-এর Variable শুধু সেই File-এর।

> **পরামর্শ:** Named Export পছন্দ করুন। এতে Import-এর নাম Export-এর নামের সঙ্গে মেলে, তাই Code-এ খোঁজা সহজ, এবং বানান ভুল হলে সঙ্গে সঙ্গে Error আসে। Default Export-এ প্রতিটি File নিজের মতো নাম দিতে পারে, যা বড় Project-এ বিভ্রান্তি তৈরি করে।

ভুল নামে Import করলে কী হয়:

wrong-import.mjs · JavaScript — চলবে: Node.js

```
import { withVAT } from '../../04-modules/tax.mjs';

console.log(withVAT(100));
```

Error Message

```
SyntaxError: The requested module '../../04-modules/tax.mjs'
does not provide an export named 'withVAT'
```

Program-এর কোনো লাইন চলার **আগেই** ত্রুটি ধরা পড়েছে। কারণ `import` স্থির (Static): JavaScript Code চালানোর আগে সব File-এর `import` ও `export` মিলিয়ে দেখে। সাধারণ Script-এ একই ভুল ধরা পড়ত শুধু সেই লাইন চলার সময়, হয়তো কোনো ব্যবহারকারীর কাছে।

১২.৫ Module-এর নিয়ম

counter.mjs · JavaScript — চলবে: Node.js

```
console.log('counter.mjs চলছে');

export let count = 0;

export function increment() {
  count += 1;
}
```

report.mjs · JavaScript — চলবে: Node.js

```
import { count } from './counter.mjs';

export const showCount = () => `report দেখছে: ${count}`;
```

main.mjs · JavaScript চলবে: Node.js

```
import { count, increment } from './counter.mjs';
import { showCount } from './report.mjs';

increment();
increment();
console.log(count, showCount());
console.log(typeof this);

try {
  total = 5;
} catch (error) {
  console.log(error.name);
}

const { withVat } = await import('../04-modules/tax.mjs');
console.log(withVat(100));
```

Output

```
counter.mjs চলছে
2 report দেখছে: 2
undefined
ReferenceError
115
```

পাঁচটি নিয়ম এই Output-এ দেখা যাচ্ছে:

১. **একবার চলে।** `counter.mjs` দুটি File থেকে Import হয়েছে, কিন্তু "চলছে" ছাপা হয়েছে একবার। প্রথম Import-এ File চলে; পরের সবাই একই Module পায়।

২. **Live Binding।** `main.mjs` ও `report.mjs` দুজনেই `count`-এর নতুন মান `2` দেখেছে। Import করা নাম মূল Variable-এর দিকে একটি জানালা, তার Copy নয়। তবে Import করা নামে সরাসরি মান বসানো যায় না (`count = 5` Error দেয়); বদলাতে হয় Module-এর নিজের Function দিয়ে।

৩. **স্বয়ংক্রিয় Strict Mode।** ঘোষণা ছাড়া `total = 5` সাধারণ Script-এ নীরবে একটি Global Variable তৈরি করত; Module-এ এটি `ReferenceError`। আর সবচেয়ে বাইরের `this` হলো `undefined`।

৪. **নিজস্ব Scope।** `main.mjs`-এর Variable অন্য কোনো File দেখে না, যতক্ষণ না সেগুলো `export` করা হয়।

৫. **Top-level `await` এবং Dynamic `import()`।** Module-এর সবচেয়ে বাইরের স্তরে `await` লেখা যায় (অধ্যায় ১০-এর প্রতিশ্রুতি)। `import()` একটি Function-এর মতো, Program চলার সময় Module আনে এবং একটি Promise ফেরত দেয়। যে অংশ সব সময় লাগে না (যেমন একটি বড় Chart Library), তা প্রয়োজনের মুহূর্তে আনতে এটি কাজে লাগে।

১২.৬ Browser-এ Module

Browser-এ Module চালাতে `<script>`-এ `type="module"` লিখুন:

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>Browser Module</title>
  <script type="module" src="app.mjs"></script>
</head>
<body>
  <h1 id="title">লোড হচ্ছে</h1>
</body>
</html>
```

greeting.mjs · JavaScript — চলবে: Browser

```javascript
export const greet = (name) => `স্বাগতম, ${name}`;
```

app.mjs · JavaScript — চলবে: Browser

```javascript
import { greet } from './greeting.mjs';

const title = document.querySelector('#title');
title.textContent = greet('Sara');

console.log(title.textContent);
console.log(typeof greet, typeof window.greet);
```

Output

```
স্বাগতম, Sara
function undefined
```

- **Module Script নিজে থেকেই `defer`।** এখানে Script `<head>`-এ, তবু `<h1>` পাওয়া গেছে (অধ্যায় ৯-এর `null` সমস্যা নেই)।
- **Global-এ কিছু যায় না।** `greet` `window`-এর Property হয়নি।
- **ঠিকানায় `./` এবং Extension বাধ্যতামূলক।** Browser `'./greeting.mjs'` বোঝে, `'greeting'` বোঝে না।
- **Server লাগে।** নিরাপত্তার কারণে Browser `file://` থেকে খোলা Page-এর Module লোড করে না; Console-এ CORS-এর ত্রুটি দেখায়। Project Folder-এ `python -m http.server 8000` বা VS Code-এর Live Server চালিয়ে `http://localhost:8000` খুলুন। (এই অধ্যায়ের সব Browser উদাহরণ একটি Local Server থেকে চালিয়ে পরীক্ষা করা হয়েছে।)

১২.৭ Node.js-এ Module

Node.js দুই ধরনের Module বোঝে। পুরোনো **CommonJS** (`require` ও `module.exports`) এখনো অনেক Package-এ আছে; নতুন Code-এ **ES Module** ব্যবহার করুন। Node.js একটি File-কে ES Module ধরে যদি:

- Extension `.mjs` হয়, অথবা
- নিকটতম `package.json`-এ `"type": "module"` লেখা থাকে (তখন `.js` File-ও ES Module)।

paths.mjs · JavaScript — চলবে: Node.js

```
import path from 'node:path';

console.log(path.basename(import.meta.dirname));
console.log(path.basename(import.meta.filename));
console.log(typeof require, typeof __dirname);
```

Output

```
08-node-paths
paths.mjs
undefined undefined
```

ES Module-এ CommonJS-এর `require`, `__dirname` ও `__filename` নেই। তাদের জায়গায় `import` এবং `import.meta.dirname` ও `import.meta.filename`। Node.js-এর নিজস্ব Module আনার সময় `node:` উপসর্গ লিখুন (`'node:path'`, `'node:fs'`), যাতে বোঝা যায় এটি Built-in, কোনো Package নয়।

পুরোনো CommonJS Code-এর সঙ্গে কাজ

বাস্তব Project-এ প্রায়ই এমন পুরোনো Code বা Package পাবেন, যা CommonJS-এ লেখা। ধরুন একটি পুরোনো Folder, যার `package.json` বলে দেয় সেখানকার `.js` File-গুলো CommonJS:

package.json · JSON — চলবে: Node.js

```
{ "type": "commonjs" }
```

tax.js · JavaScript — চলবে: Node.js

```
// পুরোনো CommonJS File: module.exports দিয়ে দেয়।
const RATE = 0.05;

function sourceTax(amount) {
  return amount * RATE;
}

module.exports = { RATE, sourceTax };
```

use-legacy.mjs · JavaScript চলবে: Node.js

```
import legacy from './legacy/tax.js';
import { sourceTax } from './legacy/tax.js';

console.log(legacy.RATE, sourceTax(2000));
```

Output

```
0.05 100
```

CommonJS File-এর `module.exports` Object-টি Default Import হিসেবে আসে, এবং Node.js যতটা পারে Named Import-ও দেয়। উল্টো পথ, অর্থাৎ CommonJS থেকে `require()` দিয়ে ES Module আনা, পুরোনো Node.js সংস্করণে কাজ করে না, এবং নতুন সংস্করণেও Top-level `await` থাকা Module-এ কাজ করে না। তাই নতুন Code সব সময় ES Module-এ লিখুন; পুরোনো Code ধীরে ধীরে রূপান্তর করুন। `package.json` ছাড়াও একটি একক File-কে CommonJS করা যায় Extension `.cjs` দিয়ে, ঠিক যেমন `.mjs` একটি File-কে ES Module করে।

১২.৮ Clean Code: পাঠযোগ্যতা আগে

Code একবার লেখা হয়, কিন্তু পড়া হয় বহুবার। নিচের Function কাজ করে, কিন্তু পড়ে বোঝা কঠিন:

before.js · JavaScript চলবে: Browser Console ও Node.js

```
function calc(o) {
  let r = 0;
  if (o) {
    if (o.items.length > 0) {
      for (const i of o.items) {
        r += i.p * i.q;
      }
      if (r > 2000) {
        r = r - r * 0.05;
      } else {
        r = r + 60;
      }
    }
  }
  return r;
}

console.log(calc({ items: [{ p: 250, q: 4 }, { p: 1850, q: 1 }] }));
console.log(calc({ items: [{ p: 120, q: 2 }] }));
```

Output

```
2707.5
300
```

একই কাজ, পরিষ্কার করে:

after.js · JavaScript — চলবে: Browser Console ও Node.js

```
const BULK_THRESHOLD = 2000;
const BULK_DISCOUNT_RATE = 0.05;
const DELIVERY_FEE = 60;

const lineTotal = (item) => item.price * item.quantity;

function orderTotal(order) {
  if (!order || order.items.length === 0) {
    return 0;
  }

  const subtotal = order.items.reduce((sum, item) => sum + lineTotal(item), 0);

  if (subtotal > BULK_THRESHOLD) {
    return subtotal * (1 - BULK_DISCOUNT_RATE);
  }
  return subtotal + DELIVERY_FEE;
}

const bigOrder = {
  items: [{ price: 250, quantity: 4 }, { price: 1850, quantity: 1 }],
};
console.log(orderTotal(bigOrder));
console.log(orderTotal({ items: [{ price: 120, quantity: 2 }] }));
```

Output

```
2707.5
300
```

ফল হুবহু এক, কিন্তু এখন Code নিজেই নিজের ব্যাখ্যা:

- **অর্থবহ নাম।** `calc`, `o`, `r`, `p`, `q` থেকে `orderTotal`, `order`, `subtotal`, `price`, `quantity`। নাম বলে দেয় **কী**, Comment ছাড়াই।
- **জাদুকরী সংখ্যার নাম।** `2000`, `0.05`, `60`-এর অর্থ এখন স্পষ্ট, এবং নিয়ম বদলালে একটি জায়গায় বদলাতে হয়।
- **Guard Clause।** তিন স্তরের `if`-এর বদলে শুরুতেই অস্বাভাবিক ক্ষেত্রটি ফেরত (অধ্যায় ৫)। মূল যুক্তি বাঁ দিকে সোজা নামে।

- **ছোট, একটি কাজের Function।** `lineTotal`-এর একটি দায়িত্ব; তার নাম পড়লেই বোঝা যায়।
- **একই ফল যাচাই।** Refactoring মানে আচরণ না বদলে গঠন বদলানো। ঠিক এ কারণেই আগে Test, তারপর Refactoring: Test প্রমাণ করে আচরণ বদলায়নি।

Comment-এর নিয়ম: Code বলে **কী** করছে; Comment বলুক **কেন**। `// subtotal-এ যোগ` অপ্রয়োজনীয়; `// সরকারি নিয়মে 2000 টাকার বেশি Order-এ ছাড় বাধ্যতামূলক` মূল্যবান, কারণ সেটি Code থেকে জানা যায় না।

১২.৯ Module সাজানো: Pure Core, Side Effect সীমানায়

একটি Application-কে Module-এ ভাগ করার ভালো নিয়ম হলো দায়িত্ব অনুযায়ী ভাগ, এবং নির্ভরতার একটি দিক:

স্তর	কাজ	নির্ভর করে
Domain (Core)	ব্যবসার নিয়ম: হিসাব, যাচাই	কিছুর ওপর নয়; Pure Function
Adapter (সীমানা)	বাইরের জগৎ: Storage, Network	Domain
UI	DOM, Event	Domain ও Adapter

Domain কখনো DOM বা `localStorage` Import করে না। এর ফলে Domain-এর প্রতিটি Function Browser ছাড়াই Node.js-এ Test করা যায়, এবং একই Domain কাল একটি Mobile App বা Server-এও কাজ করবে। অধ্যায় ৯-১১-এর Project-এ আপনি এই বিভাজন হাতে মেনেছেন; Module এটিকে File-এর গঠনে স্পষ্ট করে। Adapter-এ বাইরের জিনিসটি (যেমন `localStorage`) Parameter হিসেবে নেওয়ার কৌশলকে বলে **Dependency Injection**: Test-এ তার জায়গায় একটি নকল দেওয়া যায়, যেমন অধ্যায় ১০-এ `fetchImpl`।

Circular Import: দুটি Module পরস্পরকে চাইলে

ধরুন `cart.mjs` Import করে `storage.mjs`-কে, আর `storage.mjs`-ও Import করে `cart.mjs`-কে। JavaScript এটি অনুমতি দেয়, কিন্তু একটি Module যখন চলা শুরু করে, অন্যটি তখনো অর্ধেক চলেছে; তাই একটি Module অন্যটির `const` বা `class` তার ঘোষণার আগেই পড়তে গিয়ে `ReferenceError` পেতে পারে। কোন Module আগে চলবে তা নির্ভর করে কোন File থেকে Program শুরু হলো তার ওপর, তাই এই Bug একটি জায়গায় দেখা দেয়, অন্য জায়গায় নয়।

সমাধান গঠনগত: নির্ভরতা একমুখী রাখুন। ওপরের সারণির নিয়ম মানলে Circular Import হয় না, কারণ Domain কখনো Adapter বা UI-কে Import করে না। দুটি Module-এর যদি একই জিনিস লাগে, সেটিকে তৃতীয় একটি ছোট Module-এ সরান, যেমন Project-এ `catalog.mjs`: `cart.mjs` ও `storage.mjs` দুজনেই পণ্যের তথ্য চায়, তাই দুজনেই `catalog.mjs`-কে Import করে, পরস্পরকে নয়।

১২.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
`file://` থেকে Module Page খোলা	Console-এ CORS ত্রুটি, Page কাজ করে না	Local Server
`<script>`-এ `type="module"` নেই	`SyntaxError: Cannot use import statement outside a module`	`type="module"`
Import-এর ঠিকানায় `./` বা Extension নেই	Browser-এ ঠিকানা খুঁজে পাওয়া যায় না	`'./cart.mjs'`
Named Export-কে Default হিসেবে Import	`does not provide an export named 'default'`	`{ }`-সহ Import
Import-এর নামে বানান ভুল	`does not provide an export named ...`	Export-এর নাম হুবহু
Import করা নামে মান বসানো	`TypeError: Assignment to constant variable`	Module-এর Function দিয়ে বদলান
`.mjs`-এ `require` বা `__dirname`	`ReferenceError`	`import`, `import.meta.dirname`
দুটি Module পরস্পরকে Import করে	Import-এর সময় `undefined` বা `ReferenceError`	নির্ভরতা একমুখী রাখুন
Domain Module-এ DOM বা Storage	Node.js-এ Test করা যায় না	Adapter-এ সরান
`toSorted` ভেবে `sort`	মূল Array বদলে যায়	বদল-না-করা Method

১২.১১ Debugging নির্দেশিকা

- **Module লোড হয় না:** DevTools-এর Network Tab-এ প্রতিটি `.mjs`-এর Status দেখুন। `404` মানে ঠিকানা ভুল; File আসেনি মানে আগের কোনো Module-এ ত্রুটি।
- **Console-এর প্রথম ত্রুটি পড়ুন।** Module-এর Import-ত্রুটি Program শুরুর আগে আসে, এবং বার্তায় Module-এর নাম ও হারানো Export-এর নাম থাকে।
- **Sources Panel-এ** প্রতিটি Module আলাদা File হিসেবে দেখা যায়, এবং Breakpoint দেওয়া যায়।
- **Console-এ Module-এর Variable পাওয়া যায় না,** কারণ সেগুলো Global নয়। Breakpoint দিয়ে থামিয়ে দেখুন, অথবা সাময়িকভাবে `console.log` দিন। শুধু Debugging-এর জন্য `window.cart = cart` লেখা চলে, কিন্তু কাজ শেষে মুছে দিন।

- **Node.js-এ "Cannot use import statement outside a module"** মানে File-টি CommonJS হিসেবে চলছে: Extension `.mjs` করুন, অথবা `package.json`-এ `"type": "module"`।
- **Refactoring-এর পর আচরণ বদলেছে?** আগের ও পরের Code একই ইনপুটে চালিয়ে ফল মিলিয়ে দেখুন; এজন্যই Refactoring-এর আগে Test লিখুন।

১২.১২ পেশাদার অভ্যাস

- **একটি File, একটি দায়িত্ব;** File-এর নাম সেই দায়িত্বের নাম (`cart.mjs`, `storage.mjs`)।
- **Named Export;** Default Export শুধু যখন File-এর একটিই প্রধান জিনিস।
- **নির্ভরতা একমুখী:** UI থেকে Adapter, Adapter থেকে Domain; উল্টো দিকে কখনো নয়।
- **Domain Pure রাখুন,** বাইরের জিনিস Inject করুন।
- **নাম দিয়ে ব্যাখ্যা,** Comment দিয়ে কারণ।
- **ছোট Function, অগভীর Nesting,** Guard Clause।
- **আচরণ না বদলে ছোট ছোট ধাপে Refactoring,** প্রতিটি ধাপের পর Test।
- **একটি Code-এর ধরন (Style) পুরো Project-এ;** Prettier-এর মতো Formatter এবং ESLint-এর মতো Linter এ কাজ স্বয়ংক্রিয় করে (Tooling-এর বিস্তারিত পরের অধ্যায়গুলোতে)।

১২.১৩ Project: Modular Shopping Cart

একটি ছোট Stationery দোকানের Web Page: পণ্যের তালিকা, কার্টে যোগ, পরিমাণ বদলানো, Coupon, Delivery-র হিসাব, এবং Page বন্ধ করে খুললেও কার্ট থাকে। মূল লক্ষ্য Module-এর গঠন: প্রতিটি দায়িত্ব নিজের File-এ, এবং ব্যবসার নিয়মগুলো Browser ছাড়াই পরীক্ষাযোগ্য।

Project-এর চাহিদা

- চারটি পণ্য; একই পণ্য আবার যোগ করলে পরিমাণ বাড়ে; প্রতিটি পণ্যের পরিমাণ ০ থেকে ১০; ০ মানে সরানো।
- Coupon `STUDENT10` (১০%) ও `EID15` (১৫%); বড়-ছোট হাতের অক্ষর নির্বিশেষে; ভুল Coupon-এ বার্তা।
- ছাড়ের পর মূল্য ২০০০ টাকা বা বেশি হলে Delivery বিনামূল্যে, নইলে ৬০ টাকা; কার্ট ফাঁকা হলে Delivery নেই।
- সব টাকা পূর্ণসংখ্যায়; ছাড় নিকটতম টাকায় রাউন্ড।
- `localStorage`-এ সংরক্ষণ; ভাঙা বা অচেনা তথ্যে ফাঁকা কার্ট।
- কার্টের নিয়ম (`src/`) DOM বা Storage-এর কিছু জানে না।

File-এর কাঠামো

Folder-এর কাঠামো

```
modular-shopping-cart/
├── index.html
├── style.css
├── app.mjs                 UI: DOM ও Event
├── demo.mjs                Node.js-এ কার্টের নিয়মের প্রদর্শনী
├── src/
│   ├── catalog.mjs         পণ্যের তালিকা
│   ├── money.mjs           টাকার লেখা
│   ├── cart.mjs            কার্টের নিয়ম (Domain)
│   └── storage.mjs         localStorage Adapter
├── README.md
└── tests/
    ├── cart.test.mjs
    ├── storage.test.mjs
    ├── static-server.mjs
    ├── cdp.mjs
    └── browser-check.mjs
```

src/catalog.mjs ও src/money.mjs

catalog.mjs · JavaScript — চলবে: Browser Console ও Node.js

```javascript
// পণ্যের তালিকা; দাম পূর্ণ টাকায়।
export const PRODUCTS = Object.freeze([
  Object.freeze({ id: 'notebook', name: 'Notebook', price: 120 }),
  Object.freeze({ id: 'pen-set', name: 'Pen Set', price: 250 }),
  Object.freeze({ id: 'calculator', name: 'Calculator', price: 950 }),
  Object.freeze({ id: 'backpack', name: 'Backpack', price: 1850 }),
]);

export function findProduct(id) {
  return PRODUCTS.find((product) => product.id === id) ?? null;
}
```

money.mjs · JavaScript — চলবে: Browser Console ও Node.js

```javascript
export const formatTaka = (amount) => `${amount.toLocaleString('en-US')} টাকা`;
```

পণ্যের তালিকা `Object.freeze` দিয়ে অপরিবর্তনীয় (অধ্যায় ৭): কোনো Module ভুল করে দাম বদলাতে পারবে না। `money.mjs` একটি লাইনের File হলেও আলাদা, কারণ টাকা দেখানোর নিয়ম (যেমন পয়সা যোগ করা) বদলালে শুধু এখানেই বদলাতে হবে।

src/cart.mjs

কার্টের সব নিয়ম; দুই অংশ মিলে সম্পূর্ণ File।

cart.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
import { findProduct } from './catalog.mjs';

export const MAX_QUANTITY = 10;
export const DELIVERY_FEE = 60;
export const FREE_DELIVERY_FROM = 2000;
const COUPONS = Object.freeze({ STUDENT10: 10, EID15: 15 });

export class CartError extends Error {
  constructor(message) {
    super(message);
    this.name = 'CartError';
  }
}

// কার্ট: { items: [{ id, quantity }], coupon }; প্রতিটি Function নতুন কার্ট ফেরত দেয়।
export const emptyCart = () => ({ items: [], coupon: null });

export const isCoupon = (code) => Object.hasOwn(COUPONS, code);

export function setQuantity(cart, productId, quantity) {
  if (!findProduct(productId)) {
    throw new CartError(`অজানা পণ্য: ${productId}`);
  }
  if (!Number.isInteger(quantity) || quantity < 0 || quantity > MAX_QUANTITY) {
    throw new CartError(`পরিমাণ 0 থেকে ${MAX_QUANTITY}-এর মধ্যে পূর্ণসংখ্যা হতে হবে।`);
  }

  const others = cart.items.filter((item) => item.id !== productId);
  if (quantity === 0) {
    return { ...cart, items: others };
  }

  const exists = others.length < cart.items.length;
  const items = exists
    ? cart.items.map((item) =>
      (item.id === productId ? { ...item, quantity } : item))
    : [...cart.items, { id: productId, quantity }];
  return { ...cart, items };
}

// ...
```

cart.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
// ...

export function addItem(cart, productId, quantity = 1) {
  const current = cart.items.find((item) => item.id === productId)?.quantity ?? 0;
  return setQuantity(cart, productId, current + quantity);
}

export const removeItem = (cart, productId) => setQuantity(cart, productId, 0);

export function applyCoupon(cart, code) {
  const normalized = String(code).trim().toUpperCase();

  if (!isCoupon(normalized)) {
    throw new CartError(`Coupon "${String(code).trim()}" সঠিক নয়।`);
  }
  return { ...cart, coupon: normalized };
}

// কার্টের প্রতিটি সারি, পণ্যের নাম ও দামসহ।
export function cartLines(cart) {
  return cart.items.map(({ id, quantity }) => {
    const { name, price } = findProduct(id);
    return { id, name, price, quantity, lineTotal: price * quantity };
  });
}

export function cartTotals(cart) {
  const subtotal = cartLines(cart).reduce((sum, line) => sum + line.lineTotal, 0);
  const percent = isCoupon(cart.coupon) ? COUPONS[cart.coupon] : 0;
  const discount = Math.round((subtotal * percent) / 100);
  const afterDiscount = subtotal - discount;
  const isFree = subtotal === 0 || afterDiscount >= FREE_DELIVERY_FROM;
  const delivery = isFree ? 0 : DELIVERY_FEE;

  return { subtotal, discount, delivery, total: afterDiscount + delivery };
}
```

- **কোন নাম `export`, কোনটি নয়।** `COUPONS` বাইরে দেওয়া হয়নি; বাইরের Code জানে শুধু `isCoupon`। ভবিষ্যতে Coupon Server থেকে এলে শুধু এই File বদলাবে।
- **`setQuantity` একমাত্র দরজা।** `addItem` ও `removeItem` দুটিই এটিকে ডাকে, তাই যাচাইয়ের নিয়ম একটি জায়গায়।
- **`cartTotals` ধাপে ধাপে, প্রতিটি ধাপের একটি নাম:** `subtotal`, `discount`, `afterDiscount`, `isFree`। নিয়ম বদলালে কোন লাইন বদলাতে হবে তা নাম দেখেই বোঝা যায়।
- **`CartError`** প্রত্যাশিত ত্রুটি (অধ্যায় ১১): UI এটি ধরে ব্যবহারকারীকে বার্তা দেখাবে; অন্য সব ত্রুটি ওপরে যাবে।

src/storage.mjs

storage.mjs · JavaScript চলবে: Browser Console ও Node.js

```
import { findProduct } from './catalog.mjs';
import { emptyCart, isCoupon, MAX_QUANTITY } from './cart.mjs';

const STORAGE_KEY = 'jsm-ch12-cart';

const isItem = (item) =>
  findProduct(item?.id) !== null &&
  Number.isInteger(item.quantity) &&
  item.quantity >= 1 && item.quantity <= MAX_QUANTITY;

const isCart = (value) =>
  Array.isArray(value?.items) && value.items.every(isItem) &&
  (value.coupon === null || isCoupon(value.coupon));

// storage Inject করা যায়: Browser-এ localStorage, Test-এ একটি নকল Object।
export function createCartStore(
  storage = globalThis.localStorage,
  key = STORAGE_KEY,
) {
  return {
    load() {
      try {
        const data = JSON.parse(storage.getItem(key) ?? 'null');
        return isCart(data) ? data : emptyCart();
      } catch {
        return emptyCart();
      }
    },
    save(cart) {
      try {
        storage.setItem(key, JSON.stringify(cart));
        return true;
      } catch {
        return false;
      }
    },
  };
}
```

`createCartStore` নিজে `localStorage` ব্যবহার করে না; যা দেওয়া হয় তা-ই ব্যবহার করে। Default Parameter-এর কারণে Browser-এ কিছু না দিলেই আসল `localStorage`, আর Test-এ `getItem` ও `setItem` আছে এমন যেকোনো Object। যাচাইয়ের নিয়ম অধ্যায় ৯-এর `parseStored`-এর মতোই, তবে এখানে পণ্য ও Coupon-এর তথ্য আসে Domain Module থেকে।

Node.js-এ প্রদর্শনী

Domain Module DOM ছাড়া চলে, তাই Node.js-এ সরাসরি ব্যবহার করা যায়:

demo.mjs · JavaScript — চলবে: Node.js

```
import {
  emptyCart, addItem, applyCoupon, cartTotals, setQuantity,
} from './src/cart.mjs';
import { formatTaka } from './src/money.mjs';

let cart = emptyCart();
cart = addItem(cart, 'notebook', 2);
cart = addItem(cart, 'backpack');
console.log(cartTotals(cart));

cart = applyCoupon(cart, ' student10 ');
console.log(cartTotals(cart));

try {
  setQuantity(cart, 'notebook', 11);
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}
console.log(`পরিশোধ: ${formatTaka(cartTotals(cart).total)}`);
```

Output

```
{ subtotal: 2090, discount: 0, delivery: 0, total: 2090 }
{ subtotal: 2090, discount: 209, delivery: 60, total: 1941 }
CartError: পরিমাণ 0 থেকে 10-এর মধ্যে পূর্ণসংখ্যা হতে হবে।
পরিশোধ: 1,941 টাকা
```

দ্বিতীয় লাইনটি লক্ষ করুন: ছাড় দেওয়ার পর মূল্য ২০০০-এর নিচে নেমেছে, তাই Delivery আবার যোগ হয়েছে। নিয়মটি ("ছাড়ের পর মূল্য") ব্যবসার সিদ্ধান্ত; Code-এ তা `afterDiscount` নামে স্পষ্ট।

index.html, style.css ও app.mjs

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Modular Shopping Cart</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="app.mjs"></script>
</head>
```

```
<body>
  <main class="app">
    <h1>Stationery Shop</h1>
    <h2>পণ্য</h2>
    <ul id="products"></ul>

    <h2>কার্ট</h2>
    <ul id="cart-lines"></ul>
    <p id="empty">কার্ট ফাঁকা।</p>

    <form id="coupon-form">
      <label for="coupon">Coupon</label>
      <input id="coupon" autocomplete="off">
      <button type="submit">প্রয়োগ করুন</button>
    </form>

    <div id="totals"></div>
    <p id="message" role="status" aria-live="polite"></p>
  </main>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
  max-width: 640px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

li {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 6px 0;
}

input,
button {
  font: inherit;
```

```
  padding: 4px 8px;
}

.quantity {
  width: 4em;
}

#totals p {
  margin: 4px 0;
}
```

UI Module দুই অংশে; দুটি মিলে সম্পূর্ণ File।

app.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
import { PRODUCTS } from './src/catalog.mjs';
import { formatTaka } from './src/money.mjs';
import { createCartStore } from './src/storage.mjs';
import {
  addItem, applyCoupon, cartLines, cartTotals, CartError, setQuantity,
} from './src/cart.mjs';

const store = createCartStore();
let cart = store.load();

const productList = document.querySelector('#products');
const cartList = document.querySelector('#cart-lines');
const emptyView = document.querySelector('#empty');
const totalsView = document.querySelector('#totals');
const message = document.querySelector('#message');
const couponForm = document.querySelector('#coupon-form');

// প্রত্যাশিত ত্রুটি (CartError) বার্তা হয়; অন্য সব ত্রুটি ওপরে যায়।
function change(makeNextCart, successText) {
  try {
    cart = makeNextCart();
  } catch (error) {
    if (!(error instanceof CartError)) {
      throw error;
    }
    message.textContent = error.message;
    renderCart();
    return;
  }
  store.save(cart);
  message.textContent = successText;
  renderCart();
}
```

```
function textRow(tag, text) {
  const element = document.createElement(tag);
  element.textContent = text;
  return element;
}

// ...
```

app.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser

```
// ...

function productRow(product) {
  const button = textRow('button', 'কার্টে যোগ');
  button.type = 'button';
  button.dataset.add = product.id;

  const row = textRow('li', `${product.name}: ${formatTaka(product.price)}`);
  row.append(button);
  return row;
}

function cartRow(line) {
  const quantity = document.createElement('input');
  quantity.type = 'number';
  quantity.className = 'quantity';
  quantity.value = line.quantity;
  quantity.dataset.id = line.id;
  quantity.setAttribute('aria-label', `${line.name}-এর পরিমাণ`);

  const row = textRow('li', line.name);
  row.append(quantity, ` = ${formatTaka(line.lineTotal)}`);
  return row;
}

function renderCart() {
  const lines = cartLines(cart);
  const { subtotal, discount, delivery, total } = cartTotals(cart);

  cartList.replaceChildren(...lines.map(cartRow));
  emptyView.hidden = lines.length > 0;
  totalsView.replaceChildren(
    textRow('p', `মোট দাম: ${formatTaka(subtotal)}`),
    textRow('p', `ছাড় (${cart.coupon ?? 'নেই'}): ${formatTaka(discount)}`),
    textRow('p', `Delivery: ${formatTaka(delivery)}`),
    textRow('p', `পরিশোধ: ${formatTaka(total)}`),
  );
}
```

```
productList.addEventListener('click', (event) => {
  const id = event.target.closest('[data-add]')?.dataset.add;
  if (id) {
    change(() => addItem(cart, id), 'কার্টে যোগ হয়েছে।');
  }
});

cartList.addEventListener('change', (event) => {
  const { id } = event.target.dataset;
  change(() => setQuantity(cart, id, Number(event.target.value)), 'পরিমাণ বদলেছে।');
});

couponForm.addEventListener('submit', (event) => {
  event.preventDefault();
  const code = couponForm.elements.coupon.value;
  change(() => applyCoupon(cart, code), 'Coupon প্রয়োগ হয়েছে।');
});

productList.replaceChildren(...PRODUCTS.map(productRow));
renderCart();
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Import-এর তালিকা একটি মানচিত্র।** `app.mjs`-এর প্রথম ছয় লাইন পড়লেই বোঝা যায় এই File কোন কোন Module-এর ওপর নির্ভর করে। লম্বা Import তালিকা একাধিক লাইনে ভাঙা যায়।
- **নির্ভরতার দিক।** `app.mjs` সবাইকে Import করে; `storage.mjs` শুধু Domain-কে; `cart.mjs` শুধু `catalog.mjs`-কে; `catalog.mjs` কাউকে নয়। কোনো Domain File DOM বা Storage জানে না।
- **`change` একটি ছক:** নতুন কার্ট তৈরির চেষ্টা; `CartError` হলে বার্তা এবং কার্ট অপরিবর্তিত (তাই পরিমাণের ঘরে ভুল মান লিখলে আগের মান ফিরে আসে); সফল হলে সংরক্ষণ, বার্তা, আঁকা। নতুন কার্ট তৈরির কাজটি একটি Function হিসেবে পাঠানো হয়েছে, যাতে `try` তার ত্রুটি ধরতে পারে।
- **Event Delegation** (অধ্যায় ৯): পণ্যের তালিকায় একটি `click`, কার্টের তালিকায় একটি `change`।
- **Module Script নিজে থেকেই `defer`,** তাই `<head>`-এ থাকা সত্ত্বেও সব Element পাওয়া যায়।

চালানো এবং প্রত্যাশিত আচরণ

Project Folder থেকে:

Terminal

```
python -m http.server 8000
```

Browser-এ `http://localhost:8000` খুলুন। Test এবং Node.js-এর প্রদর্শনী:

Terminal

```
node demo.mjs
node --test
node tests/browser-check.mjs
```

কাজ	যা হওয়ার কথা
Notebook দুবার যোগ	পরিমাণ 2; মোট 240, Delivery 60, পরিশোধ 300
Backpack যোগ	মোট 2,090; Delivery 0
Coupon `student10`	ছাড় 209; ছাড়ের পর 1,881, তাই Delivery 60; পরিশোধ 1,941
Coupon `FREE100`	"Coupon "FREE100" সঠিক নয়।"; কার্ট অপরিবর্তিত
পরিমাণের ঘরে 11	সীমার বার্তা; ঘরে আগের মান ফিরে আসে
পরিমাণের ঘরে 0	সারিটি সরে যায়
Page Reload	কার্ট ও Coupon ফিরে আসে

Companion Source Code-এ ২৪টি Node.js Test আছে (কার্টের নিয়ম ও Storage Adapter), এবং `browser-check.mjs` একটি Local Server চালিয়ে আসল Chrome-এ ওপরের সব আচরণ যাচাই করে, সঙ্গে প্রমাণ করে যে `file://` থেকে খুললে Module লোড হয় না।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
পণ্যের তালিকা ফাঁকা, Console-এ CORS	`file://` থেকে খোলা	Local Server
`does not provide an export named`	Import ও Export-এর নাম মেলে না	নাম মিলিয়ে নিন
`Failed to load module script` বা `404`	Import-এর ঠিকানা ভুল	`./src/...` এবং `.mjs`
Reload-এ কার্ট হারায়	`store.save` ডাকা হয়নি	সব পরিবর্তন `change` দিয়ে
ভুল পরিমাণের পর Page থেমে যায়	`CartError` ধরা হয়নি	`change`-এর `try`
Delivery-র হিসাব অপ্রত্যাশিত	ছাড়ের আগের মূল্য ধরে হিসাব	`afterDiscount`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। দাম ও Coupon Browser-এর Code-এ আছে, তাই যেকোনো ব্যবহারকারী তা দেখতে ও বদলাতে পারেন; আসল দোকানে দাম, ছাড় ও মোট হিসাব Server আবার যাচাই করে। কোনো Checkout বা লেনদেন নেই।

১২.১৪ অনুশীলনী

প্রাথমিক স্তর

১. `[5, 1, 4]` থেকে `toSorted` দিয়ে সাজানো Copy তৈরি করুন, এবং দেখান মূল Array অপরিবর্তিত।

২. একটি Settings Object-এ `??=` দিয়ে `pageSize`-এর Default মান ২০ বসান, কিন্তু আগে থেকে `0` থাকলে তা যেন না বদলায়।

৩. `greet.mjs` থেকে Named Export `greet` এবং `main.mjs`-এ Import করে চালান।

৪. `import greet from './greet.mjs'` কেন ব্যর্থ হয়, যখন `greet.mjs`-এ শুধু `export function greet` আছে? ঠিক করুন।

৫. নিচের Function-এর নাম ও Variable-এর নাম পরিষ্কার করুন: `function f(a) { return a * 0.15; }`।

মধ্যম স্তর

৬. `structuredClone` দিয়ে একটি Nested Order-এর Copy করে Copy বদলান, এবং দেখান মূলটি অপরিবর্তিত।

৭. `Object.groupBy` দিয়ে শিক্ষার্থীদের পাস ও ফেলে ভাগ করুন (পাস নম্বর ৩৩)।

৮. একটি Module-এ `let visits = 0` এবং `recordVisit()` Export করুন; অন্য Module থেকে তিনবার ডেকে Live Binding দেখান।

৯. তিন স্তরের Nested `if` থাকা একটি Function-কে Guard Clause দিয়ে লিখুন।

চ্যালেঞ্জ স্তর

১০. Project-এর `cart.mjs`-এ একটি নতুন নিয়ম যোগ করুন: একই Order-এ ৫টির বেশি Notebook হলে প্রতিটি Notebook-এ ১০ টাকা ছাড়। শুধু Domain বদলান, এবং Node.js-এ পরীক্ষা করুন।

১১. Dynamic `import()` দিয়ে একটি "Report" Module শুধু তখনই আনুন, যখন ব্যবহারকারী চান।

১২. Project-এর `createCartStore`-এর জন্য একটি নকল Storage লিখুন, যার `setItem` সব সময় ত্রুটি ছোড়ে (জায়গা শেষ), এবং দেখান `save` তখন `false` ফেরত দেয়।

১২.১৫ অনুশীলনীর সমাধান

সমাধান ১

to-sorted.mjs · JavaScript — চলবে: Node.js

```
const numbers = [5, 1, 4];
const sorted = numbers.toSorted((a, b) => a - b);

console.log(sorted, numbers);
```

Output

```
[ 1, 4, 5 ] [ 5, 1, 4 ]
```

সমাধান ২

nullish-assign.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const fresh = {};
const custom = { pageSize: 0 };

fresh.pageSize ??= 20;
custom.pageSize ??= 20;

console.log(fresh.pageSize, custom.pageSize);
```

Output

```
20 0
```

`||=` লিখলে `custom`-এর `0`-ও ২০ হয়ে যেত।

সমাধান ৩

greet.mjs · JavaScript — চলবে: Node.js

```
export function greet(name) {
  return `আসসালামু আলাইকুম, ${name}`;
}
```

main.mjs · JavaScript চলবে: Node.js

```
import { greet } from './greet.mjs';

console.log(greet('Rafi'));
```

Output

```
আসসালামু আলাইকুম, Rafi
```

সমাধান ৪ — `import greet from` একটি **Default** Export চায়, কিন্তু File-এ আছে Named Export। Node.js বলে `does not provide an export named 'default'`। দুটি সমাধান: Import-এ বন্ধনী দিন (`import { greet } from './greet.mjs'`), অথবা File-এ `export default function greet`। Named Export-ই ভালো (অনুচ্ছেদ ১২.৪-এর পরামর্শ)।

সমাধান ৫

vat-name.mjs · JavaScript চলবে: Browser Console ও Node.js

```
const VAT_RATE = 0.15;

function vatFor(price) {
  return price * VAT_RATE;
}

console.log(vatFor(1000));
```

Output

```
150
```

সমাধান ৬

clone.mjs · JavaScript চলবে: Browser Console ও Node.js

```
const order = { id: 1, customer: { name: 'Mina', city: 'Sylhet' } };
const copy = structuredClone(order);

copy.customer.city = 'Dhaka';

console.log(order.customer.city, copy.customer.city);
```

Output

```
Sylhet Dhaka
```

সমাধান ৭

group-pass.mjs · JavaScript — চলবে: Node.js

```js
const students = [
  { name: 'Sara', marks: 88 },
  { name: 'Rafi', marks: 28 },
  { name: 'Mina', marks: 61 },
];
const PASS_MARK = 33;

const groups = Object.groupBy(students, ({ marks }) =>
  (marks >= PASS_MARK ? 'pass' : 'fail'));

console.log(groups.pass.map((student) => student.name));
console.log(groups.fail.map((student) => student.name));
```

Output

```
[ 'Sara', 'Mina' ]
[ 'Rafi' ]
```

সমাধান ৮

visits.mjs · JavaScript — চলবে: Node.js

```js
export let visits = 0;

export function recordVisit() {
  visits += 1;
}
```

main.mjs · JavaScript — চলবে: Node.js

```js
import { visits, recordVisit } from './visits.mjs';

console.log(visits);
recordVisit();
recordVisit();
recordVisit();
console.log(visits);
```

Output

```
0
3
```

`visits` Import-এর সময়ের মান নয়, মূল Variable-এর বর্তমান মান দেখায়।

সমাধান ৯

guard.mjs · JavaScript চলবে: Browser Console ও Node.js

```
function shippingLabel(order) {
  if (!order) {
    return 'Order নেই';
  }
  if (!order.paid) {
    return 'পরিশোধের অপেক্ষায়';
  }
  if (!order.address) {
    return 'ঠিকানা নেই';
  }
  return `পাঠানো হবে: ${order.address}`;
}

console.log(shippingLabel({ paid: true, address: 'Bogura' }));
console.log(shippingLabel({ paid: false }));
```

Output

```
পাঠানো হবে: Bogura
পরিশোধের অপেক্ষায়
```

প্রতিটি অস্বাভাবিক ক্ষেত্র আগেভাগে ফেরত যায়; শেষ লাইনে থাকে সফল পথ, কোনো Nesting ছাড়া।

সমাধান ১০ — Domain-এ একটি নতুন Function, এবং `cartTotals`-এর হিসাবে তার ব্যবহার। পরীক্ষার জন্য নিয়মটি আলাদা Module-এ লেখা হলো, যাতে মূল Project অপরিবর্তিত থাকে:

bulk-rule.mjs · JavaScript চলবে: Node.js

```
import {
  cartLines, addItem, emptyCart,
} from '../../project/modular-shopping-cart/src/cart.mjs';

const BULK_ITEM = 'notebook';
const BULK_MIN = 6;
const BULK_OFF_EACH = 10;

export function bulkDiscount(cart) {
  const line = cartLines(cart).find((item) => item.id === BULK_ITEM);
  return line && line.quantity >= BULK_MIN ? line.quantity * BULK_OFF_EACH : 0;
}

console.log(bulkDiscount(addItem(emptyCart(), 'notebook', 5)));
console.log(bulkDiscount(addItem(emptyCart(), 'notebook', 7)));
```

Output

```
0
70
```

"৫টির বেশি" মানে কমপক্ষে ৬টি; তাই Constant-এর নাম `BULK_MIN` এবং মান `6`। `cartTotals`-এ `discount`-এর সঙ্গে এটি যোগ করলেই নিয়মটি পুরো Application-এ কার্যকর, UI-এর একটি লাইনও না বদলে।

সমাধান ১১

report.mjs · JavaScript চলবে: Node.js

```
console.log('report.mjs লোড হলো');

export const buildReport = (sales) => `মোট বিক্রি: ${sales.reduce((a, b) => a + b,
    0)}`;
```

main.mjs · JavaScript — চলবে: Node.js

```
async function showReport(wanted) {
  if (!wanted) {
    return 'Report চাওয়া হয়নি';
  }
  const { buildReport } = await import('./report.mjs');
  return buildReport([1200, 800, 450]);
}

console.log(await showReport(false));
console.log(await showReport(true));
```

Output

```
Report চাওয়া হয়নি
report.mjs লোড হলো
মোট বিক্রি: 2450
```

প্রথম ডাকে Module লোডই হয়নি; শুধু প্রয়োজনের মুহূর্তে এসেছে।

সমাধান ১২

full-storage.mjs · JavaScript — চলবে: Node.js

```
const PROJECT = '../../project/modular-shopping-cart/src';
const { createCartStore } = await import(`${PROJECT}/storage.mjs`);
const { addItem, emptyCart } = await import(`${PROJECT}/cart.mjs`);

const fullStorage = {
  getItem: () => null,
  setItem: () => {
    throw new Error('QuotaExceededError');
  },
};

const store = createCartStore(fullStorage);

console.log(store.load().items.length);
console.log(store.save(addItem(emptyCart(), 'pen-set')));
```

Output

```
0
false
```

নকল Storage মাত্র দুটি Function; Browser ছাড়াই "জায়গা শেষ" পরিস্থিতি পরীক্ষা করা গেল। এটিই Dependency Injection-এর সুবিধা।

১২.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `[3, 1, 2].toSorted()`-এর পর মূল Array কী?

ক) `[1, 2, 3]`

খ) `[3, 1, 2]`

গ) `[]`

ঘ) `undefined`

প্রশ্ন ২. `x = 0; x ??= 5;`-এর পর `x` কত?

ক) `5`

খ) `null`

গ) `0`

ঘ) `undefined`

প্রশ্ন ৩. একটি File-এ সর্বোচ্চ কয়টি Default Export থাকতে পারে?

ক) একটি

খ) দুটি

গ) যতগুলো খুশি

ঘ) একটিও না

প্রশ্ন ৪. একটি Module তিনটি File থেকে Import হলে সেটি কতবার চলে?

ক) তিনবার

খ) একবার

গ) প্রতিটি Function ডাকার সময়

ঘ) কখনো না

প্রশ্ন ৫. Module-এর সবচেয়ে বাইরের স্তরে `this`-এর মান কী?

ক) `window`

খ) Module Object

গ) `globalThis`

ঘ) `undefined`

প্রশ্ন ৬. Browser-এ `file://` থেকে খোলা Page-এর Module লোড না হওয়ার কারণ?

ক) Browser Module সমর্থন করে না

খ) নিরাপত্তার নিয়ম (CORS); Local Server লাগে

গ) `.mjs` Extension নিষিদ্ধ

ঘ) `defer` দরকার

প্রশ্ন ৭. Node.js-এর ES Module-এ `__dirname`-এর বদলে কী?

ক) `process.cwd()`

খ) `require.main`

গ) `import.meta.dirname`

ঘ) `module.dirname`

প্রশ্ন ৮. ভুল নামে Named Import করলে ত্রুটি কখন ধরা পড়ে?

ক) Program-এর কোনো লাইন চলার আগে

খ) সেই নাম ব্যবহারের সময়

গ) কখনো না; `undefined` আসে

ঘ) Program শেষে

প্রশ্ন ৯. Clean Code-এর নিয়মে Comment কী ব্যাখ্যা করা উচিত?

ক) প্রতিটি লাইন কী করে

খ) Variable-এর ধরন

গ) Code কেন এভাবে লেখা, যা Code থেকে বোঝা যায় না

ঘ) কিছুই না; Comment নিষিদ্ধ

প্রশ্ন ১০. Domain Module-এ `localStorage` ব্যবহার না করার প্রধান কারণ?

ক) `localStorage` ধীর

খ) Domain Browser ছাড়া Test ও পুনর্ব্যবহারযোগ্য থাকে

গ) Module-এ `localStorage` নিষিদ্ধ

ঘ) Storage-এ সংখ্যা রাখা যায় না

১২.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–খ · ৫–ঘ · ৬–খ · ৭–গ · ৮–ক · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	`toSorted` নতুন Array ফেরত দেয়; মূলটি বদলায় না।
২	গ	`??=` শুধু `null` বা `undefined` -এ বসায়; `0` থেকে যায়।
৩	ক	Default Export File-এর একমাত্র প্রধান জিনিস।
৪	খ	প্রথম Import-এ চলে; পরের সবাই একই Module পায়।
৫	ঘ	Module স্বয়ংক্রিয়ভাবে Strict Mode-এ চলে।
৬	খ	Browser `file://` থেকে Module আনতে দেয় না; `http://` লাগে।
৭	গ	ES Module-এ `import.meta.dirname` ও `import.meta.filename`।
৮	ক	`import` স্থির; চালানোর আগে সব Import ও Export মেলানো হয়।
৯	গ	কী করছে তা নাম বলে; Comment বলে কেন।
১০	খ	Pure Domain Node.js-এ Test করা যায় এবং অন্য পরিবেশে চলে।

১২.১৮ অধ্যায়ের সারসংক্ষেপ

- Modern JavaScript-এর বেশির ভাগ সুবিধা আপনি আগের অধ্যায়গুলোতে শিখেছেন; নতুন: `toSorted`, `toReversed`, `with`, `||=`, `??=`, `structuredClone`, `Object.groupBy`।
- সাধারণ Script-এর Global Scope-এ নাম নীরবে সংঘর্ষে জড়ায়; Module প্রতিটি File-কে নিজস্ব Scope দেয়।
- `export` দেয়, `import` নেয়; Named Export পছন্দনীয়; Default একটিই।
- Module একবার চলে, Live Binding দেয়, স্বয়ংক্রিয় Strict Mode-এ চলে, Top-level `await` ও Dynamic `import()` সমর্থন করে।
- Import-এর ত্রুটি Program চলার আগেই ধরা পড়ে।
- Browser: `type="module"`, নিজে থেকেই `defer`, ঠিকানায় `./` ও Extension, এবং একটি Server।
- Node.js: `.mjs` বা `"type": "module"`; `require` ও `__dirname` -এর বদলে `import` ও `import.meta.dirname`।
- Clean Code: অর্থবহ নাম, নামযুক্ত Constant, Guard Clause, ছোট Function; Comment বলে কেন।
- Module সাজান স্তরে: Pure Domain, Adapter, UI; নির্ভরতা একমুখী; বাইরের জিনিস Inject করুন।

পরের অধ্যায়ে Object-Oriented Programming: Class, Prototype, Inheritance এবং Encapsulation, যা দিয়ে তথ্য ও তার আচরণ একসঙ্গে রাখা যায়।

১২.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
ES2015 (ES6)	JavaScript-এর বড় পরিবর্তনের সংস্করণ; তারপর প্রতি বছর একটি।
ES Module	`import` ও `export` ব্যবহারকারী নিজস্ব Scope-এর File।
Named Export	নির্দিষ্ট নামে Export।
Default Export	File-এর প্রধান Export; একটিই।
Namespace Import	`import * as` দিয়ে সব Export একটি Object-এ।
Live Binding	Import করা নাম মূল Variable-এর বর্তমান মান দেখায়।
Dynamic Import	`import()` দিয়ে চলার সময় Module আনা।
Top-level `await`	Module-এর বাইরের স্তরে `await`।
CommonJS	Node.js-এর পুরোনো Module-ব্যবস্থা (`require`)।
Refactoring	আচরণ না বদলে Code-এর গঠন উন্নত করা।
Guard Clause	শুরুতেই অস্বাভাবিক ক্ষেত্র থেকে ফেরত।
Dependency Injection	বাইরের নির্ভরতা Parameter হিসেবে দেওয়া।

১২.২০ আরও পড়ুন

- **MDN — JavaScript modules:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules` — Browser-এ Module-এর পূর্ণ নির্দেশিকা।
- **MDN — import:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import` — Import-এর সব রূপ।
- **Node.js — ECMAScript modules:** `https://nodejs.org/api/esm.html` — Node.js-এ ES Module, `import.meta`।
- **MDN — structuredClone():** `https://developer.mozilla.org/en-US/docs/Web/API/Window/structuredClone` — Deep Copy।
- **MDN — Object.groupBy():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/groupBy` — দলে ভাগ করা।